

Innovation Document

Aiswaria Lajan, G00486594

This document describes the technical decisions made during the development of the Conference Management system. While no additional features beyond the specification were done, several meaningful development decisions were made to ensure the application worked correctly.

Switching from mysql-connector-python to pymysql:

The original implementation used the mysql-connector-python package to connect to the MySQL database. During development and testing on a Windows machine, it was discovered that this package crashed completely silently when called : no error message, no traceback, and no output of any kind. This made debugging extremely difficult.

After isolating the issue by wrapping the connection call in a BaseException handler (which catches system-level exits as well as standard Python exceptions), the root cause was identified as a DLL compatibility issue between mysql-connector-python and the Python runtime on Windows.

The solution was to replace mysql-connector-python with pymysql, a pure Python MySQL client that does not rely on native C extensions or DLLs. This resolved the issue immediately. The cryptography package was also required as a dependency for SHA256 password authentication used by MySQL on the ATU Virtual Machine.

Packages required:

- pymysql: replacement for mysql-connector-python
- neo4j: official Neo4j Python driver
- cryptography: required for MySQL SHA256 authentication

Install command:

```
pip install pymysql neo4j cryptography
```

Room Data Caching:

The project specification says that any room added to the MySQL database after option 6 has been called for the first time should not appear until the application is restarted. To implement this, a global variable `_rooms_cache` was used.

On the first call to `view_rooms()`, the data is fetched from MySQL and stored in `_rooms_cache`. On every subsequent call during the same session, the cached data is displayed instead of querying the database again. This is a straightforward implementation of session-level caching that directly meets the specification requirements.

Cross-Database Lookups in Options 4 and 5:

Options 4 and 5 required careful coordination between the MySQL and Neo4j databases. The key decisions were:

- MySQL is always checked first before any Neo4j operation. This ensures that nodes are never created in Neo4j for attendees that do not exist in the relational database, which is a requirement stated in the specification.

- The Neo4j MERGE statement is used when creating connections. MERGE creates a node only if it does not already exist, which automatically handles the case where an attendee is in MySQL but not yet in Neo4j.
- The CONNECTED_TO relationship is queried in both directions using an undirected pattern (-[:CONNECTED_TO]-) so that connections show up regardless of which direction they were created in.
- After retrieving connected attendee IDs from Neo4j, a second MySQL query is used to look up each attendee's name. This keeps Neo4j responsible only for the graph relationships, while MySQL remains the source of truth for attendee details.

Input Validation and Error Handling:

Throughout the application, input validation was implemented carefully to match the error conditions described in the specification:

- Company IDs are validated to be positive integers before any database query is made. Non-numeric and negative inputs are silently rejected, and the user is prompted again.
- Gender is validated as Male or Female before the INSERT is attempted, rather than relying on the database ENUM constraint to reject it. This produces a cleaner error message.
- Duplicate attendee ID errors (MySQL error 1062) are caught separately from other database errors so the correct message format from the specification can be shown.
- All other MySQL errors (invalid ID format, invalid date format, etc) are passed through directly from the database as per the specification.